



HTML/CSS II

講義資料 - 後期



HTML/CSS II

実習9 【改変】 PHPを使ってみよう

今回の内容

1. MAMP環境のインストール
2. PHPのセットアップ
3. PHPの文法振り返りとProgateの補講
4. テストページを作成してみよう

✅ 後期で扱う言語やソフトウェアなどは、例外事項も多いため基本の解説に留める

MAMPとは？



(M)ac(A)pache(M)ySQL(P)HP

Mac環境で、「Apache」、「MySQL」、「PHP」を使うことができる環境のこと。

- Apache: ウェブサーバーソフトウェア
- MySQL: DBを管理するシステム、混在しやすいが言語としてのSQLとは別
- PHP: サーバーサイドで動作するプログラミング言語

業務としての普段使いには向かない可能性がある

HTMLやJavaScriptではブラウザに依存するため、最新を中心にブラウザ対応することが必要だった。
PHPやDBの場合は提供サイドの環境に依存するため、AMPそれぞれのバージョンが動作に直結する。
サーバーもLinuxで動かしている場合が多く、本番サービスの開発環境としては注意が必要。

MAMP環境のインストール

MAMPをインストールしよう

MAMPのインストール手順

<https://www.mamp.info/en/mac/>

- フリーダウンロード
- 解凍してインストーラを起動(完了後インストーラなどは破棄してOK)
- MAMPを起動し、Apacheを起動
- MAMP(フリー版)を起動し、「Start」でサイトが立ち上がればOK
 - `http://{ドメイン}:{ポート}/uri`

マルチ運用の設定をしよう

ヴァーチャルホストの設定(マルチドメイン運用) - hosts

MAMPのフリー版ではマルチドメイン設定に対応していない。

手動でマルチサイトを可能にするために、次の作業を行う。

- ホストの書き換え: 「/private/etc/hosts」
 - ターミナルでファイルを確認「sudo cat /private/etc/hosts」 ※PCパスワードが求められたら入力
 - ファイルの確認ができたなら実際に編集「sudo vi /private/etc/hosts」
 - 「i」キーでインサートモードにして、「127.0.0.1 mysite.localhost」を記述後、「ESQ」→「:wq」→「Enter」
 - ※間違えたと思ったら、「:q!」で編集を破棄できる(quit+強制終了)
 - 上書きをチェックするため、再度「sudo cat /private/etc/hosts」

✔管理者権限を意味する「sudo」コマンドは乱用しないこと

マルチ運用の設定をしよう

ヴァーチャルホストの設定(マルチドメイン運用) - MAMPのVirtualHosts

hostsで名前解決ができるようにしているが、MAMP側では配信側の設定をする。

ローカル側の閲覧設定が「hosts」で、サーバー側のマルチ配信設定を「VirtualHosts」で行う

- ヴァーチャルホストを有効化: 「/Applications/MAMP/conf/apache/httpd.conf」をテキストで編集
 - 下の方にある「Include /Applications/MAMP/conf/apache/extra/httpd-vhosts.conf」の記述をコメントアウト
- ヴァーチャルホストの書き換え: 「/Applications/MAMP/conf/apache/extra/httpd-vhosts.conf」をテキストで編集
 - 上記のファイルを開く(テキストエディタでOK)
 - <VirtualHost>~</VirtualHost>までをコピー
 - 下記サンプルを参考にサイトの設定を記述

```
<VirtualHost *:8888>
```

```
ServerAdmin webmaster@dummy-host2.example.com
```

```
DocumentRoot "/Applications/MAMP/htdocs/mysite.localhost"
```

```
ServerName mysite.localhost
```

```
ErrorLog "logs/mysite.localhost-error_log"
```

```
CustomLog "logs/mysite.localhost-access_log" common
```

```
</VirtualHost>
```

マルチ運用の設定をしよう

ヴァーチャルホストの設定(マルチドメイン運用) - 独自ドメインでのサイト動作を確認

ヴァーチャルホストで設定したディレクトリにファイルを作成し、ブラウザからアクセスを確認する。

- 「/Applications/MAMP/htdocs/mysite.localhost/index.php」を作成
- 以下に示す確認用のコードを記述
- ブラウザで接続確認「http://localhost:8888/mysite.localhost/」
 - ドメインなどをカスタマイズしている場合は適宜変更する
- 大量の文字が出ればOK。PHPの設定などが書いてある

```
<?php
```

```
echo "表示テスト<br>";
```

```
phpinfo();
```


PHPの文法ポイント

拡張子と記述方法

- 拡張子: 「.php」
- 記述ルール: 「<?php」で開始して、「?>」で閉じる
 - HTMLとの組み合わせが可能: <p>私は<?php echo \$name; ?>です</p>

JavaScriptと比較したPHPの特徴

- サーバーサイドで配信前に書き換えを行うため、クライアント側からソースは確認できない
- エラーを発生させると500エラーのリスクがある
- 動作環境はブラウザではなくサーバーに依存する
- プログラム上の比較ポイント
 - 変数は「\$name」のように「\$」を付けて宣言する
 - 特別な宣言や型情報は必須ではない
 - 四則演算は一緒だが、文字列の連結は「.」で行う
 - 「elseif」の「else」と「if」の間にスペースは付けない
 - ループや条件は「:」と「end」記述が可能

PHPの文法ポイント

実際のコードを見て確認しよう

```
sup.php > ...
1  <?php
2
3  $num = 100 + 100 * 10 / 5 % 7;
4  $name = '田中' . '太郎';
5  echo '計算結果は: ' . $num . '<br>';
6  echo '名前は: ' . $name . '<br>';
7
8  if ($num > 10000) {
9  } elseif ($num > 100) {
10 } else {
11 }
12
13 if (true):
14 | // ここにifの結果
15 else:
16 | // ここにelseの結果
17 endif; //if-endif ここまで
```

PHPの文法ポイント

配列の使い方を学ぼう

【配列】

データを「,」区切りで記述し、0から始まる添え字で記述する方式。JavaScriptの配列とあまり大きな違いはないが、PHPの場合はデータ型。

【連想配列】

文字列を添え字とした配列で、「=>」でインデックスとデータをつなぐ。JavaScriptのオブジェクトに近いが、考え方が違うため1:1で対比はできない。

```
// ----- 配列

$arr = [
    100,
    200,
    300,
    400,
    500
];
echo $arr[0] . $arr[4];

// ----- 連想配列

$strArr = [
    'id' => 1,
    'name' => '田中',
    'address' => 'test@example.com',
];

echo 'ID(' . $strArr['id'] . ')の方の名前は「' . $strArr['name'] . '」で、メールアドレスは「' . $strArr['address'] . '」を使っている。」;
```

PHPの文法ポイント

forループの使い方を学ぼう

JavaScriptとほとんど同じのため、簡単な解説に留める。

if文で解説した通り、中括弧でくくる場合と「:」で区切る場合がある(HTML混合で見やすい)。

```
// ----- forループ
for ($i = 0; $i < 5; $i++) {
    echo $arr[$i] . '<br>';
}
echo '<hr>';
for ($i = 0; $i < 5; $i++):
    echo $arr[$i] . '<br>';
endfor;
```

100

200

300

400

500

100

200

300

400

500

PHPの文法ポイント

whileループの使い方を学ぼう

「特定の条件に合った場合に、ループをさせる記法。

for文は回数ループと相性が良く、whileは条件ループと相性が良い。

while-endwhileも同様に使うことができる(同じことの繰り返しになるため結果は省略)。

```
// ----- whileループ
$i = 0;
while ($i < 5) {
    echo $arr[$i] . '<br>';
    $i++;
}

$i = 0;
while ($i < 5):
    echo $arr[$i] . '<br>';
    $i++;
endwhile;
```


PHPの文法ポイント

foreachループの使い方を学ぼう

指定した配列やオブジェクトが末端に到達するまでループさせる。
連想配列やオブジェクト型との相性が良い。

```
// ----- foreachループ
foreach ($arr as $var) { // 取り出した要素を「$var」と名付ける
    echo $var . '<br>';
}
// 添え字部分を取り出すこともできる(key => value形式),
// 配列の場合は配列の添え字の整数
foreach ($strArr as $key => $str):
    echo $key . ': ' . $str . '<br>';
endforeach;
```

```
100
200
300
400
500
id: 1
name: 田中
address: test@example.com
```

PHPの文法ポイント

breakとcontinue

- break: ループ中に実行された場合ループを終了する
- continue: ループ中に実行された場合、ループ内の残りの処理を破棄し、次のループを実行に移る

```
// continue, break
for ($i = 0; $i < 5; $i++) {
    if ($i === 2) {
        break;
    }
    echo $arr[$i] . '<br>';
}
echo '▼▼▼▼' . '<br>';
for ($i = 0; $i < 5; $i++) {
    if ($i === 2) {
        continue;
    }
    echo $arr[$i] . '<br>';
}
```

100

200



100

200

400

500

PHPの文法ポイント

シンプルな関数の記述

```
function {関数名}({引数}) {  
    // 値を返す場合  
    return {結果};  
}
```

```
// ----- シンプルな関数  
function simpleFunc($v1 = '') {  
    return '結果は: ' . $v1 . '<br>';  
}  
echo simpleFunc('変数に渡す値');
```

結果は変数に渡す値

テストページを作成してみよう

例題

自分の名前を変数にして表示してみよう

以下のコードの「\$name」の部分を、PHPで正しく表示できるようにする。

```
<!DOCTYPE html>
<html>
<head>
  <title>PHPの実行サンプル</title>
  <meta charset="utf-8" />
  <meta name=" viewport" content=" width=device-width,initial-scale=1">
</head>
<body>
  <?php $name = '田中<br>'; ?>
  <h1>私の名前は$nameです。 </h1>
</body>
</html>
```

テストページを作成してみよう

例題

PHPのセキュリティを考えてみよう

例題のプログラムを正しく実行すると、変な位置で改行が入る。

実際の運用に当てはめると、ユーザーによるフォーム入力のが実行できてしまう可能性がある。

「htmlspecialchars」という関数でエスケープ処理ができるので、変数を出力する場合は活用しよう。

<https://www.php.net/manual/ja/function htmlspecialchars.php>

✔ 特殊文字を HTML エンティティに変換する…PHP上で意味のある文字列を通常のテキストとして変換すること

✔ 前の例題のプログラムに記述して結果を確かめてみよう

✔ ページのソースを見てみると、エスケープ処理の実装内容が確認できる



HTML/CSS II

実習10【改変】フォームを実装してみよう

今回の内容

- 1.2-1 PHPで動的な値を使おう
- 2.2-2 安全な使い方を考えてみよう
- 3.2-3 フォームを作ってみよう

2-1 PHPで動的な値を使おう

動的な値の使い方

PHPタグをHTMLタグと混ぜて使う

「sample1/index.php」 コピーして、動的な値の使い方について確認する。

- GETパラメーター: URLの「URL?値1のラベル=値1&値2のラベル=値2&...」
- POSTパラメーター: URLからは見えないので、プログラム内で受け取る
 - 見ようと思えば見えるので安全とは思わないこと

GETやPOSTなどの仕様はサイトへ。

<https://www.php.net/manual/ja/language.variables.external.php>

値を入れて変化を確かめてみよう。

2-2 安全な使い方を考えてみよう

PHPの直接実行

セキュリティリスクを考える

フォームのテキストボックスに以下の記述を入れてみよう。

```
echo "<script>alert('alt')</script>"
```

PHPを直接実行させないための方法

エスケープ処理というものを加えて、ユーザーの入力をただのテキストとして処理するように記述する。
いくつか方法はあるが、`filter_input`などは表示に、保存には`htmlspecialchars`が代表的。

「example2/index.php」を開いて動作を確かめてみる。

```
echo "<script>alert('alt')</script>"
```

2-3 フォームを作ってみよう

フォームを実装してみよう

「sample3」を参考にフォームを実装する

コメントで処理の流れを記述しているため、それにしたがってプログラムを記述する。
難易度が高めのため、調べながらチャレンジしてみることに。

※セッションについて

サーバーに一時的に値を保存しておき、同じユーザーがアクセスした時に配信する仕組み。
この仕組みによって、通信で失われてしまう情報を保持できる。
会員サイトなどでログイン情報が引き継げるのは、セッションなどの仕組みが理由。

ちなみに、ログイン状態の保持はセッションではない。



HTML/CSS II

実習11 【改変】 データベースとフォームを組み合わせてみよう

今回の内容

- 1.3-1 データベースを理解しよう
- 2.3-2 データベースを作成してみよう
- 3.3-3 掲示板風のプログラムを作ってみよう

データベースの構造

データを効率的に管理するシステム、管理イメージは表計算ソフト(例外あり)

データの編集や検索に特化したシステム。通常のファイルで起きるような同時書き込みや冗長的な検索などは発生しにくく、動的な要素のないサイト以外にはほぼ必須のシステム。

■主要なデータベース

- MySQL(MariaDB) - WordPressの標準
- PostgreSQL
- MongoDB(NoSQL)

ID	名前	メールアドレス
1	田中 太郎	tanaka@example.com
2	佐藤 花子	sato@example.com

3-1 データベースを理解しよう

データベースの概形図

■データベース(RDBMS)

■テーブルA(会員)

3列×2行のテーブル

会員ID	名前	メールアドレス
1	田中 太郎	tanaka@example.com
2	佐藤 花子	sato@example.com

■テーブルB(チャット履歴)

4列×3行のテーブル

チャットID	会員ID	チャット内容	投稿日時
1	1	新年が明けました	2000/1/1
2	1	猫の日です	2010/2/2
3	2	3000年まであと1日	2999/12/31

主要なデータベースを理解する

RDBMS (リレーショナルデータベース)

前項のように、特定のカラム(列)の情報を元に、関連(リレーション)する情報を紐づけて取得できるDBをRDB(リレーショナルデータベース)と呼ぶ。

サイトなどでよく使われる形式のため、RDBに絞って取り扱う。

主キー

RDBでは、特定のレコード(行)を定めるため、**テーブル内で1つだけ存在する値として主キー**が必要となる。

チャット履歴のように同じユーザーが複数登録するなど、一意に定めることが難しい場合はIDとして連番の数値を用いる。

- チャット履歴の場合: 投稿数に合わせて連番で用意される無意味の値
- 会員の場合: ユーザー名またはメールアドレスなど元々重複が許されない値を用いることが可能

データ型

他のプログラミング言語と同様に、いくつかの型が存在する。主要なものは、「数値」、「文字列」、「日付・日時」、「バイナリ」などスクリプト言語に比べると制約が細かい。

3-2 データベースを作成してみよう

phpMyAdminを使ったDB管理

phpMyAdminとは？

データベースをブラウザ上などで管理するためのシステム。コマンドを使うより直感的。

MAMPからphpMyAdminを起動しよう

- MAMPの「WebStart」でブラウザからページを起動
- メニューの「Tools」から「phpMyAdmin」を起動
- デフォルトユーザーとしてログインした状態でアクセスされる
 - ユーザー名: root
 - パスワード: root
 - ホスト(接続先): localhost:8889



3-2 データベースを作成してみよう

データベースを作ってみよう

以下のコマンドでデータを作成する

phpMyAdminのメニューから「SQL」を選択(データベースは選択しないこと)。
gitの最新情報を取得し、「mysite.localhost/chapter3/db/db_init.sql」のコードを実行する。

ファイルに書いてあるSQLを実行しよう

SQLの記述欄にコードの全てをコピーペースト。
メニュー上のリロードマークを押し、「sample_chat」というデータベースと、「chat_history」というテーブルができていればOK。

3-3 掲示板風のプログラムを作ってみよう

サンプルコードを書いてみよう

サンプルコードを自分で書いてみる

実際に手を動かしてみて、シンプルなデータベースとの連携プログラムを書いてみよう。

gitの最新情報を取得し、「mysite.localhost/chapter3/example1,2,3.php」の「chapter3」以下のコードを自分の環境に書く。

※phpの動く場所は「/Applications/MAMP/htdocs/mysite.localhost/」で固定されているので、chapterのディレクトリを作るようにしよう。

ファイル名はそれぞれ、以下に書き換えて動作できるように変更する。

- sample1.php => index.php
- sample2.php => edit.php
- sample3.php => complete.php

もし時間が余ったら、編集ボタンの近くに削除ボタンを作成し、データを消せるようにしよう。

書式: DELETE FROM テーブル名 WHERE 条件;

参考: <https://www.javadrive.jp/mysql/insert/index10.html>



HTML/CSS II

実習12【移動】WordPressをインストールしてみよう

今回の内容

1.4-1 WordPressとは？

2.4-2 WordPressをローカル環境にインストールしよう

4-1 WordPressとは？

CMSを理解しよう

CMSとWordPressとは？

深い知識がなくても、Webサイトなどを管理、編集できるようにするシステム。

最も世界で使われているCMSがWordPressである。

ウェブサーバー、php、データベースで構成され、オープンソースで開発されている。

なぜWordPressを使うのか

- ベースを作っておけばエンジニアに依存せずにサイトを管理できる
- メニューなどの共通部分をパーツ化することで、効率的な管理ができる
- プラグインが豊富で拡張が簡単
- 無料のプラグインで十分構成できる

WordPressを使う上での注意点

- phpとデータベースを使うためセキュリティーにより対策が必要
- みんなが使っているためBot攻撃などの標的には狙われやすい
- メンテナンスをしていないと上記の理由で改ざんや踏み台にされることがある

4-1 WordPressとは？

WordPressの構成

■ファイル(FTP)

- システムデータ
- テンプレートのファイル(テーマ)
- システムの設定ファイル
- プラグインデータ

■データベース

- 管理者情報
- プラグインを含む各種設定(ex: ページネーションの表示数やURLなど)
- 各投稿データ
- パーツデータ

WordPressは拡張性が高く様々な構成があるが、システムデータはファイルとして、その後追加されたデータはおよそデータベースに管理される。

✓ 通常のサイトとは異なり、サイトデータだけを持って行ってもページとして表示できない

✓ 「WordPressを移行」しようと思った場合は、必ずデータとデータベースの両方がなければならない

4-2 WordPressをローカル環境にインストールしよう

公式のアプリを使おう

WordPress Studio をインストールしよう

<https://developer.wordpress.com/ja/studio/>

インストールが終わったら、日本語名から「my-first-test」などにしておこう。

✓ 詳細情報から保存先を確認しておこう

1. Studioの画面から、サイトや管理画面を開いてみよう
2. 投稿や固定ページを作成し、実際に表示してみよう
3. プラグイン「Yoast SEO」を入れてみよう
 - a. 投稿や固定ページは変わっているだろうか
4. 設定でパーマリンクを変えてみよう
 - a. 投稿を増やしてエディタに慣れてみる
5. 日本のビジネステーマ「lightning」をインストールしてテーマを変えてみよう
 - a. 推奨プラグインを入れてみよう
 - b. ソースコードでテーマの違いを見よう(クラシックテーマ、ブロックテーマ)
6. 過去の投稿を見つつ、新しい投稿を作成してみよう
7. ウィジェットを変更してサイトの状態を確認してみよう



HTML/CSS II

実習13 【移動】 WordPress(クラシックテーマ)を使ってみよう

今回の内容

1.5-1 クラシックテーマとは？

2.5-2 PDFファイルを元にクラシックテーマを作成してみよう

5-1 クラシックテーマとは？

クラシックテーマの基本

クラシックテーマとは？

登場からしばらく使われていたテーマ。

現在公式が推奨しているテーマは「ブロックテーマ」。

phpとHTMLの複合で記述し、書いたものがそのまま反映されるシンプルな構成。

→WordPress特有の書き方があまりないため確認しやすい(WordPress特有の関数はある)

5-1 クラシックテーマとは？

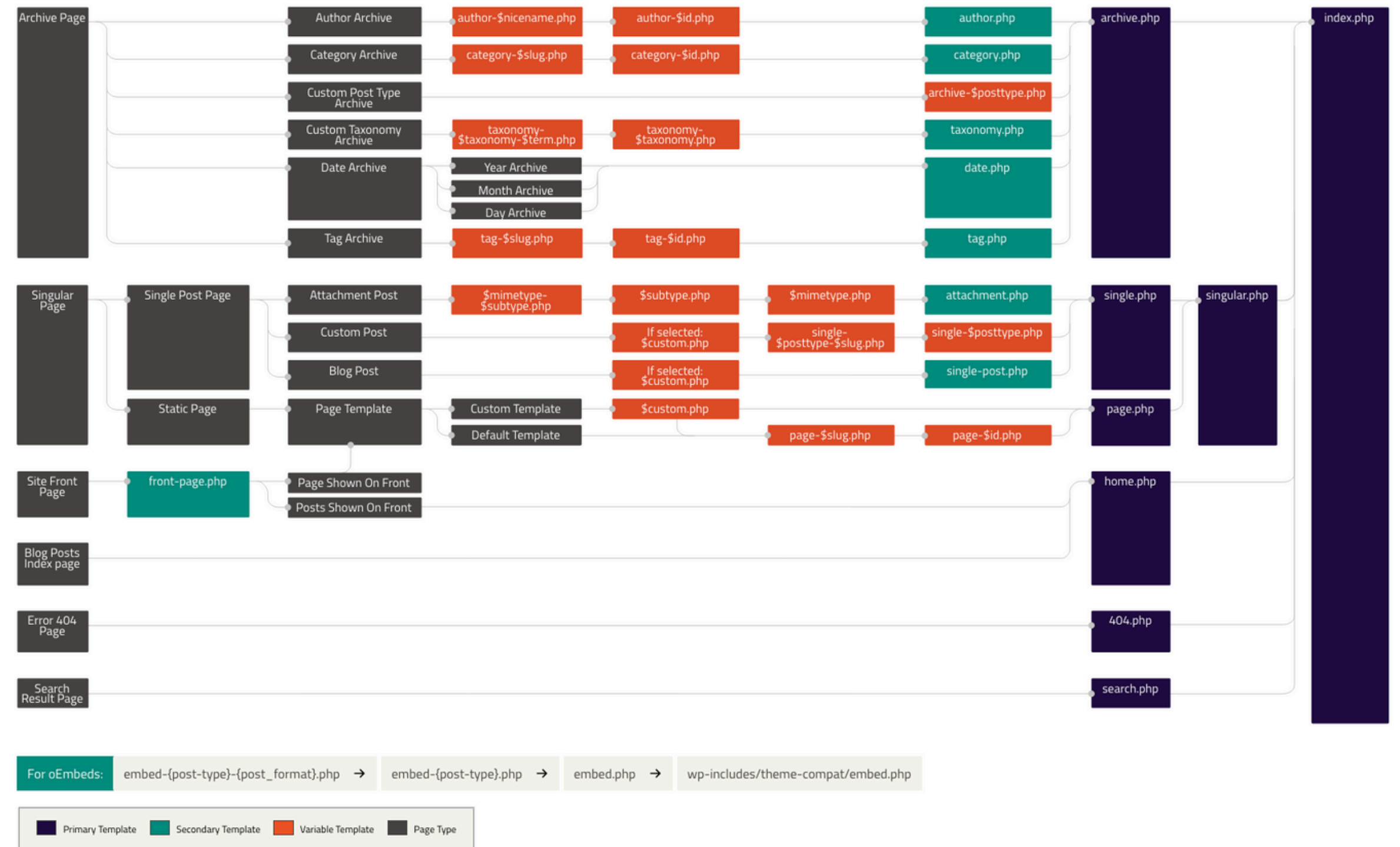
クラシックテーマの優先度

アーカイブファイルの優先度

WordPressではどのテンプレートをしようするかが自動的に選択される。

ファイルがなくエラーという状況を防ぐことができる機構で、投稿あるいはそのカテゴリー、固定ページなど、どれを使うかを考えて用意する。

共通デザインならまとめてしまってもOK。



5-2 PDFファイルを元にクラシックテーマを作成してみよう

ファイルの役割を捉えよう

手順書に従ってクラシックテーマを構成する

gitリポジトリからファイルを更新し、「[introduction/wordpress/classic.pdf](#)」を順番に進める



HTML/CSS II

実習14 【移動】 WordPress(クラシックテーマ)を使ってみよう 2

今回の内容

1.6-1 クラシックテーマをカスタマイズしよう

6-1 クラシックテーマをカスタマイズしよう

ファイルの役割を捉えよう

手順書に従ってクラシックテーマを構成する

gitリポジトリからファイルを更新し、「[introduction/wordpress/classic-custom.pdf](#)」を順番に進める



HTML/CSS II

実習15 【移動】 WordPress(クラシックテーマ)を使ってみよう3

今回の内容

- 1.6-1 納品を想定したWordPress構成を考えよう
- 2.6-2 納品前のチェックリストを用意しよう
- 3.6-3 設定予定のプラグインについて

6-1 納品を想定したWordPress構成を考えよう

納品できる状態を想定しよう

必要な状態の完成図を考えよう

WordPressに限らず、納品(完成)状態にするまでにはあるべき形というものがある。
個人にしても企業にしても必要なものは異なるが、以下の観点は最低限考えておこう。

- きちんと完成した状態になっているか
 - エラー状態も確認する
- 修正や更新が対応できる環境になっているか
- セキュリティ対策はされているか
- バックアップ体制はできているか
- SEO対策が十分か

6-2 納品前のチェックリストを用意しよう

テーマチェックリスト

※チェックリストを考える上での注意点

画一的な正解は正解はなく、ここに提示したのも一例である。
個別のプロジェクトや企業ルールを優先すること。

目標物を決めて、プラグインを決めて、テーマやカスタマイズを行う

前回までの構築で行った通り、プラグインを前提としたカスタマイズが度々登場する。
したがって、最終的な成果物を先に決めて、プラグインを入れた後初めてテーマの調整に入る順番となる。
既存テーマのカスタマイズもそれなりに制約があるため、テーマでデザインをざっくり決めて、カスタマイズ項目を確認した後に提案すると変更量は低く抑えることも可能。
カスタマイズの範囲に制限がつくので、顧客の調整ができるなら納品までのスピードを向上できる。
✅ テーマでできる範囲でほとんどのカスタマイズを済ませてしまう。

6-2 納品前のチェックリストを用意しよう

テーマチェックリスト

テーマの完成チェックリスト

- テーマは完成しているか
- 子テーマなどを使って、アップデートされても消失しない状態になっているか
- カテゴリー、タグ、スラッグの設定は問題ないか
- リンク切れは発生していないか
- HTMLとしての構成に問題はないか(ですます調の統一など基本的なことも含めて)
- 子テーマのスクリーンショットが設定されているか

6-2 納品前のチェックリストを用意しよう

サイト設定リスト

サイト設定リスト

- ファビコンが設定されているか
- タイトルやディスクリプションなどの設定が変更できるようになっているか
- OGPが必要か、設定がページごとに反映されるか
- JSON-LDなどは必要か、設定されているか
- RSSフィードなどの外部連携は適切か

6-2 納品前のチェックリストを用意しよう

デバッグリスト

デバッグリスト

- デバッグモードでエラーがないか
- 管理画面のサイトヘルスは出していないか
- meta robotsは正常か(サイトのインデックス状態)
- (フォームがある場合)メールは送信されるか、DNSゾーン設定など
 - SPF/DKIM/DMARC(対応の場合)

6-3 設定予定のプラグインについて

プラグインリスト

設定予定のプラグインの種類(細かくは変更を踏まえてPDFに記述)

- セキュリティ
- SEO
- お問い合わせ、リダイレクト、リキャプチャ、保存
- リダイレクトプラグイン
- 移行ツール
- 画像圧縮ツール&キャッシュ

本来は入れるがカットするプラグイン

- GTM(登録しないため)
- エディター補助
- リキャプチャ(登録しないため)

6-3 設定予定のプラグインについて

プラグインリスト

規模や体制によって考慮すべきプラグイン

- バックアッププラグイン
 - レンタルサーバーで標準がある場合は不要と思われる
 - バックアッププラグインと自動バックアップの相性はよく考える
- ワークフロープラグイン
 - 上長との区分け、本部・支部体制では必要とされやすい
- 権限プラグイン
 - 管理画面を細かく権限分けしたい時、設定が複雑化するのでその点は注意

6-3 設定予定のプラグインについて

実際にサイトを作ってみよう

PDFを元にサイトを構成しよう

「[introduction/wordpress/classic-comp.pdf](#)」にしたがって作業を進めよう



HTML/CSS II

実習16ブロックテーマを使ってみよう

今回の内容

- 1.7-1 コンテナ型の Wordpress仮想環境「Local」を導入しよう
- 2.7-2 ブロックテーマの基本2025年版のソースコードを確認してみよう
- 3.7-3 オリジナルのテーマを組み上げてみよう

7-1 コンテナ型の Wordpress仮想環境「Local」を導入しよう

Localの導入

Localとは？(旧 local by flywheel)

Dockerコンテナベースで動くWordPress用の仮想環境。
クリックでサイトを構築ができ、本番さながらの環境を再現できる。
MAMPに比べて複数サイトが容易な他、他のプラットフォームでも動作する。

Localをインストールしよう

以下のサイトからダウンロードし、インストールまで実行する。
メールアドレスは必要。
名前は空欄でOK。
<https://localwp.com/>

Localによるサイト管理

サイトの追加・管理手順

1. 新規追加

- a. サイトが0の場合、中央のCreateボタンから生成する
- b. サイトがある場合、サイドメニューのプラスボタンから

2. サイトの追加手順

- a. アルファベットでサイトネームを入力する(my-block-theme など)
- b. 環境はおすすめでいいが、実際は運用状況に合わせる
- c. ユーザー名、パスワードはローカルで使う分には単純でOK
 - i. 追加オプションで日本語を選択する
- d. サイト追加で追加される、権限が要求されたら許可する

7-1 コンテナ型の Wordpress仮想環境「Local」を導入しよう

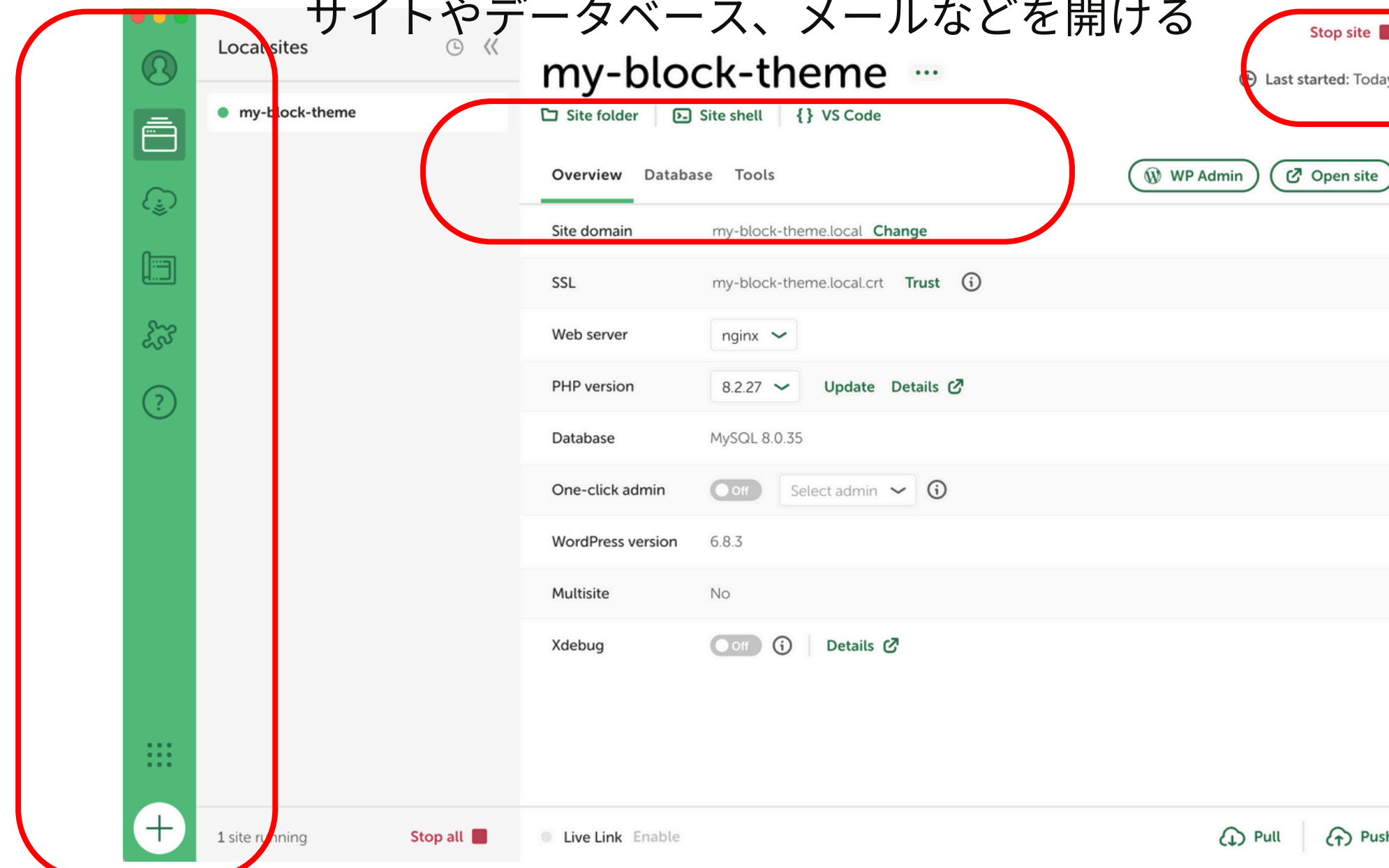
サイト管理を見てみよう

■サイドメニュー
ローカル環境では
あまり使わない

■メインタブ

サイトやデータベース、メールなどを開ける

■サイトの起動・停止



7-2 ブロックテーマの基本2025年版のソースコードを確認してみよう

2025年版の公式テーマ

Twenty Twenty Fiveブロックテーマ

サイトデータは以下の場所にある。

環境のためのファイルが上層にあるので消さないように注意する。

「/Local Sites/{{ドメイン名}}/app/public」

データ状況はデフォルトのため、テーマは以下の位置。

「/Local Sites/{{ドメイン名}}/app/public/wp-content/themes/」

立ち上げた初期状態で有効化されているので、管理画面とコードを比較しながら確認する。

クラシックテーマと同じように、最低限のファイルは多くない。

- style.css
- templates/index.html

7-3 オリジナルのテーマを組み上げてみよう

空のテーマを作成しよう

テーマを少しずつ組み上げていこう

「[introduction/wordpress/block-first.pdf](#)」にしたがって作業を進めよう